

哈喽各位亲爱的宝子，我是里昂！很高兴认识你。

接下来学长将会从考研真题出发，浓缩考点，以尽可能清晰易懂的方式带你搞懂大题套路！

课程使用指南

1. 本课程包含了知识点回顾、基操训练、真题解析、习题训练
2. 大题阶段以练题为主，请大家务必多动手，多思考

接下来开始今天的快乐学习！

注意，**大题强化课程主要是针对408统考**，408大题并不会涉及到较难的算法，**考察的基本都是偏基础的内容。**

大题强化课程并非单纯的大题求解，而是顺带会把一些相关的知识点给大家复习了，即结合大题的二轮强化复习。

另外，本课程为了照顾基础一般/跨考的宝子，会有大量偏基础的内容；对于基础较好的宝子，可以选择跳过/倍速过。

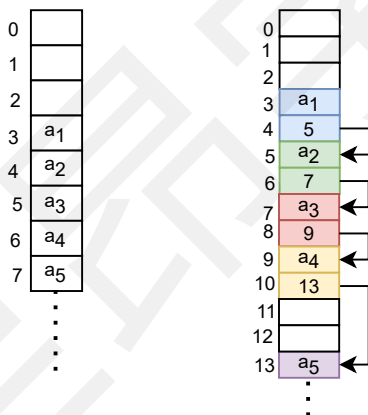
一、线性表

线性表是具有相同数据类型的数据元素的一个有限序列。

线性表L通常表示为： $(a_1, a_2, a_3, \dots, a_i, a_{i+1}, \dots, a_n)$ ，其中n指的是序列中元素的个数，n为0时，表明线性表为空。

除了第一个以及最后一个元素以外，**其他元素都有且仅有唯一的前驱以及后继。**

线性表是逻辑结构，表明了数据之间的关系，但其物理结构不唯一，**可以通过顺序存储结构或者链式存储结构实现。**



(一) 知识点回顾（偏基础，理解并记忆）

1. 顺序表的简单实现

顺序表通常用数组来实现。

【例】现在定义了一个int型数组，五个数组元素分别是{100,1000,8,15,200}，请将其循环打印。（跟我一起写一遍）
(对于基础好的宝子可以跳过该例题，主要是让基础一般的宝子逐渐适应手写代码)

```
#include<stdio.h>

int main(){
    //创建了一个数组
    int array[5]={100,1000,8,15,200};
    //循环打印每个数据元素
    for(int i=0;i<5;i++){
        printf("a[%d]=%5d",i,array[i]);
    }
    return 0;
}
```

手写区（左手捂答案，右手写完对照）

注：408真题可以不写头文件，这里主要是照顾自命题或者想要上机的同学

注：C语言薄弱的同学最好还是先学习下C语言基础课程

实际考试时，不管是选择题，还是代码大题，基本上直接用上述那种最简单的方式实现就行，即定义一个数组。
而官方定义主要有静态分配和动态分配两种。

2. 顺序表的官方定义(理解即可)

静态分配实现定义如下：

```
#include<stdio.h>
#define MAX_SIZE 100 //定义顺序表最大长度
typedef int ElemType; //定义数据元素类型
typedef struct{
    ElemType data[MAX_SIZE]; //存放顺序表中的元素
    int length; //顺序表当前存放的元素的个数，即表长
}SqList; //顺序表的类型定义

int main(){
    //创建了一个线性表
    SqList sq;
    sq.length=5;
    sq.data[0]=100;
    .....
}
```

手写区（左手捂答案，右手写完对照）

typedef是一个关键字，作用是**为现有的数据类型起别名，目的就是方便！**

基本格式：**typedef 已有变量类型 别名**
typedef int elemtype;
elemtype a = 10; // 等同于 int a = 10;

结构体以struct关键词开头

在C语言中，结构体是一种自定义的复合数据类型，可以包含多个不同类型的变量和数据成员。

结构体的作用在于将一组相关的变量和数据聚合在一起，更好地组织程序结果或数据。

```
struct student {
    int id;
    int age;
    float height;
};
{ }中列出结构体中包含的变量或者成员
```

动态分配实现定义如下：

```
.....
#define INIT_SIZE 100 //表长度的初始定义
typedef struct{
    ElemType *data; //指示动态分配数组的指针
    int maxsize,length; //数组的最大容量和当前个数
}SqList; //动态分配数组顺序表的类型定义

int main(){
    //创建了一个线性表
    SqList sq;
    sq.maxsize=INIT_SIZE;
    sq.length=5;
    sq.data=(ElemType*) malloc(sizeof(ElemType)*INIT_SIZE);
    //malloc函数动态申请内存空间
    sq.data[0]=100;
    .....
}
```

手写区（左手捂答案，右手写完对照）

3. 顺序表的基本操作

这里我们用静态分配下的顺序表示例，熟练掌握以下基本操作即可。

3.1 查找操作

在顺序表L中查找一个值为e的元素，并返回其位序，e是要查找的元素的值

```
int locateElem(SqList L, ElemType e){
    for(int i=0; i<L.length; i++){
        if(L.data[i]==e)
            return i; //找到则返回数组下标
    }
    return -1; // -1表示查找失败
}
```

手写区（左手捂答案，右手写完对照）

3.2 插入操作

18	25	6	98	7	100	95		60	68		45	50
0	1	2	3	4	5	6		i	i+1		n-2	n-1

长度为n的数组

若要在下标为i的地方插入一个元素，则意味着要将i~n-1的所有元素都向后移动一个位置

18	25	6	98	7	100	95			60	68		45	50
0	1	2	3	4	5	6		i	i+1	i+2		n-1	n

长度为n+1的数组

随后在i处插入新元素49

18	25	6	98	7	100	95		49	60	68		45	50
0	1	2	3	4	5	6		i	i+1	i+2		n-1	n

```
//在下标为i处插入一个元素
bool insertElem(SqList &L, int i, ElemType e){
    if(i<0||i>L.length) return false; //i的合法范围为[0, L.length]
    if(L.length>=MAX_SIZE){
        return false; //若顺序表已满，则插入失败
    }
    for(int j=L.length; j>i; j--){
        L.data[j]=L.data[j-1];
        //从最后一个元素一直到下标为i的元素依次往后移动
    }
    L.data[i]=e; //下标为i处插入e
    L.length++;
    return true;
}
```

手写区（左手捂答案，右手写完对照）

```
#include <stdio.h>
int main() {
    int x = 10;
    int y=x;
    int& ref = x;
    printf("x的值为: %d\n", x);
    printf("y的值为: %d\n", y);
    printf("ref的值为: %d\n", ref);
    // 通过引用修改变量的值
    ref = 20;
    printf("修改后, x的值为: %d\n", x);
    printf("修改后, y的值为: %d\n", y);
    printf("修改后, ref的值为: %d\n", ref);
    return 0;
}
```

引用

C++是对C语言的继承，引用（reference）就是C++对C语言的扩充。

在C++中，引用可以理解为一个已存在变量的另一个名字。

其格式：变量类型 & 引用名称 = 变量名

C:\Users\Administrator\Desktop

```
x的值为: 10
y的值为: 10
ref的值为: 10
修改后, x的值为: 20
修改后, y的值为: 10
修改后, ref的值为: 20
```

3.3 删除操作

```
//删除下标为i处的元素，用e记录被删除元素的值
bool deleteElem(SqList &L,int i,ElemType &e){
    if(i<0||i>=L.length) return false;//i的合法范围为[0,L.length)
    e=L.data[i];//下标为i处的值赋值给e
    for(int j=i;j<L.length-1;j++){
        L.data[j]=L.data[j+1];//i后面的元素依次向前移动
    }
    L.length--;
    return true;
}
```

3.4 交换操作

```
bool swapElem(int a[], int i,int j){
    int temp=a[i];
    a[i]=a[j];
    a[j]=temp;
}
```

手写区（左手捂答案，右手写完对照）

4. 指针

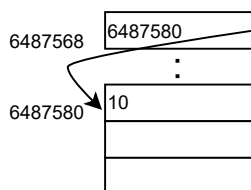
指针本质上也是一类变量类型，只不过其存储的不是具体的数据，而是某个内存单元的地址。

```
#include<stdio.h>
int main(){
    int x=10;
    int* y=&x;
    printf("x=%d\n",x);
    printf("x的地址=%d\n",&x);
    printf("y=%d\n",y);
    printf("y的地址=%d\n",&y);
}
```

手写区（左手捂答案，右手写完对照）

int 作为一种整型变量，其代表的存储区域内有一个整数。

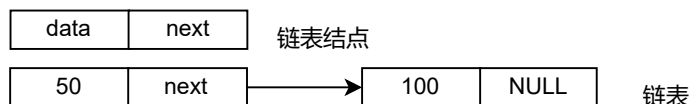
int* 作为一种指针型变量，其代表的存储区域内有一个地址，这个地址指向了一个int型的整数。



```
C:\Users\Administrator\Desktop
x=10
x的地址=6487580
y=6487580
y的地址=6487568
```

5. 链表

下面是一个简单链表的逻辑结构图，每个链表结点由data域和next域组成，该链表共有两个链表结点。



下面是该链表的C语言实现代码。

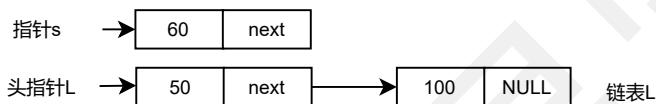
```
.....
typedef struct LNode{
    int data;
    struct LNode* next;
}LNode;
int main(){
    LNode* p=(LNode *)malloc(sizeof(LNode));
    p->data=100;
    p->next=NULL;
    LNode* q=(LNode *)malloc(sizeof(LNode));
    q->data=50;
    q->next=p;
    .....
```

手写区 (左手捂答案, 右手写完对照)

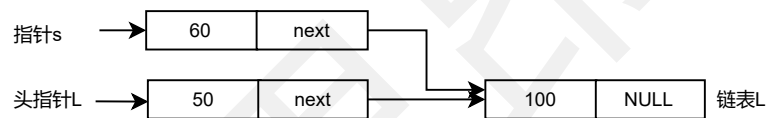
6. 链表的基本操作

6.1 插入操作

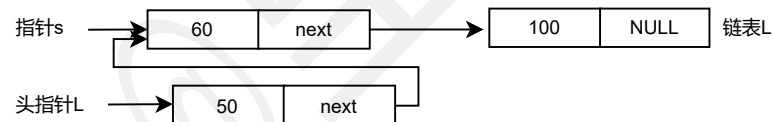
现想将指针s指向的结点插入链表L中, 头指针L是记录链表的第一个结点地址的指针变量。



首先将s指向的结点的next域修改, 将里面的地址修改成L->next, 即s->next=L->next;



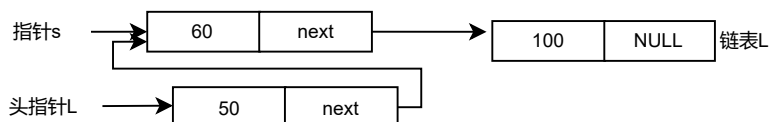
接着将L指向的结点的next域修改, 将里面的地址修改成s, 即L->next=s;



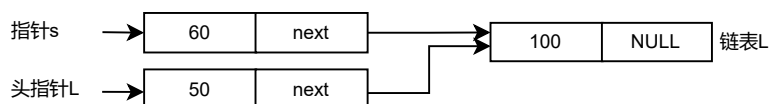
若从一个单链表的头部插入, 则称为**头插法**; 若从尾部插入, 则称为**尾插法**。

6.2 删除操作

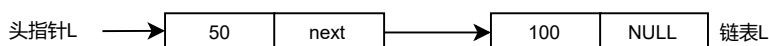
现想将指针s指向的结点从链表L中删除, 头指针L是记录链表的第一个结点地址的指针变量。



首先将L指向的结点的next域修改, 将里面的地址修改成s->next, 即L->next=s->next;



接着将s指向的结点所占用的内存空间释放, 即free(s)。



手写区 (左手捂答案, 右手写完对照)

手写区 (左手捂答案, 右手写完对照)

(二) 基操训练

【例1】设计一个高效算法，将**顺序表L**的所有元素**逆置**，要求算法的**空间复杂度为 $O(1)$** 。

步骤1 分析题目条件，理解题目意思：

- ① 顺序表
- ② 空间复杂度在 $O(1)$ ，往往意味着不开辟额外数组，不进行递归调用
- ③ 具体的操作：逆置

思想：化抽象为具象，通过一个具体的例子，让自己明白是什么意思，例如 1, 2, 3, 4==>4, 3, 2, 1

步骤2 解题模板（优先暴力解）

- ① 确定数据结构类型：顺序存储实现的线性表（顺序表）
- ② 回忆顺序表的特点：
 - 1) 顺序存储，随机存取
 - 2) 插入元素，需要依次后移多个元素
 - 3) 删除元素，需要依次前移多个元素
 - 4) 交换元素

步骤3 题目要求拆解

- 1) 不能额外开辟数组
- 2) 首尾元素互换

步骤4 答题模板

(1) 算法思想如下：

- ① 扫描顺序表L的前半部分元素
- ② 对于元素L.data[i]($0 \leq i < L.length/2$),将其与后半部分的对应元素L.data[L.length-i-1]进行交换。

算法思想分步骤写，写清楚每步都做了什么。

(2) 算法代码如下：

```
void Reverse(SqList & L){
    ElemType temp;//辅助变量
    for(int i=0;i<L.length/2;i++){
        //交换L.data[i]与L.data[L.length-i-1]
        temp=L.data[i];
        L.data[i]=L.data[L.length-i-1];
        L.data[L.length-i-1]=temp;
    }
}
```

手写区（左手捂答案，右手写完对照）

(3) 题目没有明确要求的话，可以不写时空复杂度。

【例2】从**有序顺序表**中**删除所有其值重复**的元素，使表中所有元素的值均不同。

步骤1 分析题目条件，理解题目意思：

- ① 有序顺序表
- ② 没有时空复杂度要求
- ③ 具体的操作：删除所有值重复

思想：化抽象为具象，通过一个具体的例子，让自己明白是什么意思，例如 1, 2, 2, 2, 3, 4==>1, 2, 3, 4

步骤2 解题模板（优先暴力解）

步骤3 题目要求拆解

- 1) 有序意味着重复的元素是连续的
- 2) 若当前元素与前驱元素不同，则删除后续与其值相同的元素
- 3) 删除通过前移实现

步骤4 答题模板

(1) 算法思想如下：

- ① 初始时将第一个元素视为非重复元素，用变量*i*记录当前非重复元素，*i*=0
- ② 用变量*j*记录第一个与L.data[*i*]不同的元素，*j*=1
- ③ 不断比较L.data[*i*]与L.data[*j*]，若相等，则 *j*++；
若不相等，则将L.data[*j*]前移至L.data[*i*]后面，两者同时向后移动一个位置
- ④ 更新顺序表长度

(2) 算法代码如下：

```
void DeleteSame(sqList& L){
    int i,j; //i指向第一个不相同的元素，j为查找指针
    for(i=0,j=1;j<L.length;j++)
        if(L.data[i]!=L.data[j])//查找下一个与上个元素值不同的元素
            L.data[++i]=L.data[j];//找到后，将元素前移
    L.length=i+1;
}
```

手写区（左手捂答案，右手写完对照）

另外，还可以通过辅助数组的方式实现，有兴趣的同学可以尝试下。

【例3】设计一个算法用于判断带头结点的循环双链表是否对称。